Linnest Typst API

Linnest exposes the `linnest.wasm` graph layout plugin and the `linnest-curve.wasm` Bezier helper plugin through a small Typst API. The public surface is intentionally narrow:

- `graph` for construction, parsing, inspection, joins, and graph algorithms.
- `subgraph` for subgraph object construction and inspection.
- `curve` for Bezier splitting and CeTZ curve emission.
- `layout` for the separate layout pass.
- `draw` for rendering a laid-out graph object with CeTZ.

Minimal Builder Example

```
#import "../src/lib.typ": curve, draw, graph, layout, subgraph

#let b = graph.builder(
  name: "demo",
  edge-statements: (
    eval_label: "(text(fill: rgb(\"#{color}\"))[{label}])",
    eval_source: "(stroke: rgb(\"#{source-color}\") + 0.5pt)",
    eval_sink: "(stroke: rgb(\"#{sink-color}\") + 0.5pt)",
  ),
)
#let (node: a, builder: b) = graph.node(b, name: "a")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let b = graph.edge(
  b,
  source: (node: a, compass: "e"),
  sink: (node: c, compass: "w"),
  statements: (
    color: "0055ff",
    source-color: "d72638",
    sink-color: "1b7f4c",
    label: "a-c",
  ),
)
#let g = graph.finish(b)
#let g = layout(g, seed: 2, steps: 5)
#let east = subgraph.compass(g, "e")
#let edges = graph.edges(g, subgraph: east)
#let dot = graph.dot(g)
#draw(g, subgraph: east)
```

Graph Objects

Graph, builder, and subgraph objects are opaque zero-copy values. Build or parse graph objects with `graph`, transform graph objects with `layout`, and pass objects back to `graph` or `subgraph` for inspection.

- `graph.parse(input)` parses one or more DOT digraphs and returns an array of graph objects.
- `graph.build(..)` constructs one graph object from named arguments. This is convenience sugar over the builder functions.
- `graph.builder(..)` starts a builder object.
- `graph.node(builder, ..)` returns `(node: index, builder: builder)`.
- `graph.edge(builder, ..)` returns the updated builder.
- `graph.finish(builder)` turns a builder into a graph.
- `layout(graph, seed: 2, steps: 5, ..)` runs layout as an explicit second step. Its settings are named parameters so calls stay descriptive and Tidy can document each field.
- `draw(graph, ..)` draws a laid-out graph object with CeTZ.
- `graph.dot(graph)` returns a DOT string for inspection or export.

Builder Spec

The builder API is the preferred construction API when nodes need to be reused. Use destructuring to keep the builder value moving:

```
#let (node: a, builder: b) = graph.node(b, name: "a")
```

`graph.edge` accepts `source` and `sink` endpoint dictionaries. An endpoint can contain `node`, `statement`, `id`, `port_label`, `compass`, and `in_subgraph`. Set `source: none` or `sink: none` to create an external half edge.

String-valued DOT statements may contain `{name}` placeholders. `graph.build` and the builder object expand each placeholder while constructing the graph object. Use `{{` and `}}` for literal braces. Unknown placeholders are left unchanged.

This is most useful for graph-level `edge-statements`: the default edge statement is merged into each edge, then expanded against that edge's complete statement dictionary. The following default label renderer consumes the per-edge `label` statement:

```
#let b = graph.builder(
  edge-statements: (
    eval_label: "(text(fill: rgb(\"#{color}\"))[{label}])",
    eval_source: "(stroke: rgb(\"#{source-color}\") + 0.5pt)",
    eval_sink: "(stroke: rgb(\"#{sink-color}\") + 0.5pt)",
  ),
)
#let (node: a, builder: b) = graph.node(b, name: "a")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let b = graph.edge(
  b,
  source: (node: a),
  sink: (node: c),
  statements: (
    color: "0055ff",
    source-color: "d72638",
    sink-color: "1b7f4c",
    label: "a-c",
  ),
)
```

`graph.build` accepts the same data in one dictionary:

```
#let g = graph.build(
  name: "demo",
  statements: (full_num: "x + y"),
  node-statements: (shape: "circle"),
  edge-statements: (
    color: "000000",
    eval_label: "(text(fill: rgb(\"#{color}\"))[{label}])",
  ),
  nodes: ((name: "a"), (name: "b")),
  edges: ((
    source: (node: 0, compass: "e"),
    sink: (node: 1, compass: "w"),
    statements: (color: "0055ff", label: "ab"),
  ),),
)
```

Graph Queries

`graph.info(g)` returns graph metadata. `graph.nodes(g)` returns node records, and `graph.edges(g)` returns edge records. Pass `subgraph: sg` to filter nodes or edges by an subgraph object.

`graph.join(left, right, key: "statement")` joins matching dangling half edges. The key is read from half-edge data and can be `"statement"`, `"port_label"`, `"compass"`, or `"id"`.

`graph.cycles(g)` returns subgraph objects for a cycle basis. `graph.forests(g)` returns subgraph objects for spanning forests.

Layout Model

`layout` starts from a traversal-tree placement, then optimizes the positions of graph nodes and edge control points. The initial tree spacing is

$$L = \lambda \sqrt{\frac{WH}{\max(n, 1)}}$$

,

with horizontal spacing $\tau_x L$ and vertical spacing $\tau_y L$. Here λ is `length_scale`, W is `viewport_w`, H is `viewport_h`, τ_x is `tree_dx`, and τ_y is `tree_dy`. These fields set the geometry scale for both layout modes.

The shared spring/charge model uses the following coefficients:

$$c_{\mathrm{vv}} = \beta L^2$$

$$c_{\mathrm{ev}} = \beta \gamma_{\mathrm{ev}} L^2$$

$$c_{\mathrm{ee}} = \beta \gamma_{\mathrm{ee}} L^2$$

$$c_{\mathrm{center}} = \beta g_{\mathrm{center}} L^2$$

$$c_{\mathrm{dangling}} = \beta \gamma_{\mathrm{dangling}} L^2$$

The Typst parameter names are `beta` for β, `gamma_ev` for γ_{ev}, `gamma_ee` for γ_{ee}, `g_center` for g_{center}, and `gamma_dangling` for $\gamma_{\mathrm{dangling}}$. The spring stiffness k is `k_spring`, and the softening constant ε is `eps`.

In `layout_algo: "anneal"`, linnest minimizes an energy:

$$E = \sum_{i<j} \frac{1}{2} \frac{c_{\mathrm{vv}}}{d(v_i, v_j) + \varepsilon} + \sum_{i,e} \frac{c_{\mathrm{ev}}}{d(v_i, e) + \varepsilon} + \sum_{(v,e)\ \text{incident}} \frac{1}{2} k(\ell_e - d(v, e))^2$$
$$+ \sum_{\text{local edge pairs}} \frac{1}{2} \frac{c_{\mathrm{ee}}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling pairs}} \frac{1}{2} \frac{c_{\mathrm{dangling}}}{d(e_i, e_j) + \varepsilon} + \sum_i \frac{1}{2} \frac{c_{\mathrm{center}}}{d(v_i, 0) + \varepsilon} + p_{\mathrm{cross}} N_{\mathrm{cross}}$$

.

Here p_{cross} is `crossing_penalty` and N_{cross} is the number of detected edge crossings. `temp`, `step`, `seed`, `steps`, `epochs`, `cool`, `accept_floor`, `step_shrink`, and `incremental_energy` belong to this simulated annealing mode. `crossing_penalty` is also anneal-only; force mode does not currently add a crossing force.

In `layout_algo: "force"`, linnest applies the direct forces corresponding to the same vertex-vertex, edge-vertex, incidence spring, local edge-edge, dangling-edge, and center terms. `step` is the integration step, `delta` clamps per-step movement, `steps` and `epochs` set the iteration budget, `cool` shrinks the step after each epoch, and `early_tol` stops when movement is small. `z_spring` and `z_spring_growth` are force-only helpers: the integrator gives points temporary z coordinates to break overlaps and pulls them back toward the 2D plane.

`directional_force` is applied in both modes as an extra bias derived from pin/port direction constraints.

After either graph layout mode, labels are relaxed separately. If L_l is the label target distance and q_l is the label repulsion strength, then $L_l = \alpha_l L$ and $q_l = \beta_l L^2$. The Typst names are `label_length_scale` for α_l and `label_charge` for β_l. `label_spring` is the spring constant pulling each label toward its target. `label_steps`, `label_step`, `label_early_tol`, and `label_max_delta_scale` control the label relaxation iteration.

Subgraphs

Subgraph objects are opaque zero-copy values.

- `subgraph.label(g, label)` constructs a subgraph from a base62 label.
- `subgraph.bits(g, bits)` constructs a subgraph from a boolean hedge array.
- `subgraph.compass(g, compass)` selects half edges with a DOT compass point.
- `subgraph.to-label(sg)` returns the base62 label.
- `subgraph.hedges(sg)` returns included hedge indices.
- `subgraph.contains(sg, hedge)` tests hedge membership.

Curve API

The `curve` module is the Typst facade for `linnest-curve.wasm`. It keeps the graph drawing layer small by providing reusable Bezier helpers:

- `curve.split-cubic` splits one cubic Bezier at parameter `t`.
- `curve.trim-cubic` and `curve.trim-segment` trim a cubic by arc length from either endpoint. `draw` uses this to start edges at the node outset instead of at the node center.
- `curve.split-through` builds a quadratic through a layout point and returns two cubic halves.
- `curve.hobby-through` builds the default smooth edge geometry through the source node, edge layout point, and sink node.
- `curve.edge-halves` combines `hobby-through` and trimming for graph edges.
- `curve.cetz-bezier`, `curve.cetz-pattern`, and `curve.cetz-edge-halves` convert returned geometry to CeTZ drawing commands.

All geometric points passed to `curve` are dictionaries with numeric `x` and `y` fields. Cubic segments use:

```
#let segment = (
  start: (x: 0, y: 0),
  ctrl_a: (x: 1, y: 0.5),
  ctrl_b: (x: 2, y: -0.5),
  end: (x: 3, y: 0),
)
```

Pattern Objects

`curve.pattern-cubic` and `curve.pattern-segment` generate a one-dimensional pattern along a cubic path. Built-ins are plain Typst pattern dictionaries:

```
#let w = curve.wave()
#let z = curve.zigzag()
#let c = curve.coil(longitudinal-scale: 1.6)
```

String names such as `"wave"`, `"zigzag"`, and `"coil"` are accepted for convenience, but they are resolved in Typst before the wasm plugin is called. Custom patterns use the same dictionary shape:

```
#let hook = (
  kind: "points",
  name: "hook",
  interpolation: "linear",
  points: (
    (at: 0, x: 0, y: 0),
    (at: 0.25, x: 0.15, y: 1),
    (at: 0.5, x: 0, y: 0),
    (at: 0.75, x: -0.15, y: -1),
    (at: 1, x: 0, y: 0),
  ),
)
```

`at` is the phase position within one wavelength, from `0` to `1`. `x` offsets along the path tangent and `y` offsets along the path normal; both are scaled by the pattern amplitude. If `at` is omitted, points are spaced evenly. Use `interpolation: "linear"` for sharp corners and `"smooth"` for a smooth spline through sampled points.

`endpoint_ramp: true` tapers the pattern amplitude at anchored endpoints. This is useful for `coil`, whose longitudinal offset would otherwise make the first and last visible points sit inside the turn near nodes.

```
#let path = curve.pattern-segment(
  segment,
  pattern: hook,
  amplitude: 0.15,
  wavelength: 0.7,
)
#cetz.canvas({
  curve.cetz-pattern(path, stroke: black + 0.5pt)
})
```

Generated Reference

- layout()

Variables

- graph
- curve
- subgraph

layout

Apply the linnest layout pass to a graph object.

This is intentionally a second step: construct or parse a graph first, then call `layout`. Set `layout_algo` to `"force"` for deterministic force integration or `"anneal"` for simulated annealing.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#let g = layout(g, seed: 2, steps: 5)
#graph.edges(g).map(edge => edge.pos)
```

```
((x: 0.22939424194475896, y:
0.8844117105825495),)
```

Parameters

```
layout(
  graph: bytes,
  viewport-w: float,
  viewport-h: float,
  tree-dx: float,
  tree-dy: float,
  steps: int,
  seed: int,
  step: float,
  step-shrink: float,
  cool: float,
  accept-floor: float,
  early-tol: float,
  temp: float,
  delta: float,
  beta: float,
  k-spring: float,
  g-center: float,
  epochs: int,
  crossing-penalty: float,
  gamma-dangling: float,
  gamma-ee: float,
  directional-force: float,
  label-length-scale: float,
  label-spring: float,
  label-charge: float,
  label-steps: int,
  label-step: float,
  label-early-tol: float,
  label-max-delta-scale: float,
  gamma-ev: float,
  eps: float,
  incremental-energy: bool,
  layout-algo: string,
  z-spring: float,
  z-spring-growth: float,
  length-scale: float
) -> bytes
```

graph `bytes`

Graph object returned by `graph.build`, `graph.finish`, or `graph.parse`.

viewport-w `float`

Width of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `10.0`

viewport-h `float`

Height of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `10.0`

tree-dx `float`

Horizontal spacing multiplier for the initial traversal-tree placement. Applies before both `"force"` and `"anneal"`.

Default: `0.9`

tree-dy `float`

Vertical spacing multiplier for the initial traversal-tree placement. Applies before both `"force"` and `"anneal"`.

Default: `1.2`

steps `int`

Iterations per epoch. In `"force"` mode this is the number of force integration steps; in `"anneal"` mode this is the number of proposals per temperature epoch.

Default: `int(sys.inputs.at("steps", default: "15"))`

seed `int`

Seed for deterministic initialization, force-mode jitter, and annealing proposals. Applies to both modes.

Default: `int(sys.inputs.at("seed", default: "14"))`

step `float`

Initial movement scale. `"force"` multiplies computed forces by this value; `"anneal"` uses it as the proposal step size.

Default: `0.81`

step-shrink `float`

Anneal-only step shrink factor, applied when an epoch's acceptance ratio falls below `accept_floor`.

Default: `0.21`

cool `float`

Cooling factor applied once per epoch. `"anneal"` cools `temp`; `"force"` shrinks `step`.

Default: `0.85`

accept-floor `float`

Anneal-only acceptance-ratio threshold below which `step` is shrunk by `step_shrink`.

Default: `0.15`

early-tol `float`

Force-only early stop threshold for maximum movement in one step. The annealing schedule stores this value but does not currently use it for stopping.

Default: `1e-6`

temp `float`

Anneal-only initial temperature used in the Metropolis acceptance test.

Default: `0.3`

delta `float`

Force-mode maximum movement clamp per point and per step.

Default: `0.4`

beta `float`

Base repulsion strength for vertex-vertex interactions. Also scales `gamma_ev`, `gamma_ee`, `gamma_dangling`, and `g_center`. Applies to both modes through the shared spring energy.

Default: `46.1`

k-spring `float`

Spring stiffness for node-to-edge incidence lengths. Applies to both modes.

Default: `11.0`

g-center `float`

Centering strength relative to `beta`. Applies to both modes.

Default: `40.0`

epochs `int`

Number of epochs. Both modes run up to `steps` iterations inside each epoch.

Default: `30`

crossing-penalty `float`

Anneal-only fixed energy penalty per detected edge crossing. The direct force integrator does not currently add a crossing force.

Default: `30.0`

gamma-dangling `float`

Repulsion for dangling half edges, relative to `beta`. Applies to both modes through the shared spring energy.

Default: `40.0`

gamma-ee `float`

Local edge-edge repulsion, relative to `beta`. Applies to both modes.

Default: `0.1`

directional-force `float`

Bias that pushes points in directions implied by pin/port constraints. Applies to both modes.

Default: `5.0`

label-length-scale `float`

Edge-label target offset as a multiple of the graph spring length. Label layout runs after both graph layout modes.

Default: `0.6`

label-spring `float`

Spring strength pulling each label toward its target offset. Label layout runs after both modes.

Default: `23.0`

label-charge `float`

Repulsion strength between labels and graph points, scaled by spring length squared. Label layout runs after both modes.

Default: `3.0`

label-steps `int`

Maximum number of post-layout label relaxation steps. Set to `0` to skip label placement. Applies after both modes.

Default: `20`

label-step `float`

Label relaxation step size. Applies after both modes.

Default: `0.15`

label-early-tol `float`

Label relaxation early stop threshold. Applies after both modes.

Default: `1e-3`

label-max-delta-scale `float`

Label movement clamp as a multiple of spring length. Applies after both modes.

Default: `0.5`

gamma-ev `float`

Edge-vertex repulsion, relative to `beta`. Applies to both modes.

Default: `0.1`

eps `float`

Softening epsilon used in inverse-square force/energy terms. Applies to both modes.

Default: `1e-4`

incremental-energy `bool`

Whether annealing updates cached energy by local deltas. This is anneal-only; force mode computes direct forces instead of energies.

Default: `true`

layout-algo `string`

Layout algorithm. Use `"force"` for direct force integration or `"anneal"` for simulated annealing against the spring energy.

Default: `"force"`

z-spring `float`

Force-only spring pulling temporary z coordinates back toward the layout plane. Use with `z_spring_growth` to help separate overlapping points during integration.

Default: `0.05`

z-spring-growth `float`

Force-only per-epoch multiplier for `z_spring`.

Default: `1.3`

length-scale `float`

Natural spring-length multiplier. This scales the graph's preferred edge length and the repulsive coefficients derived from it. Applies to both modes.

Default: `0.1`

graph `module`

Graph namespace.

Graph objects are opaque zero-copy values. Keep them opaque and pass them back to this module, `subgraph`, or `layout()`.

curve `module`

Curve namespace.

Helpers for splitting and drawing Bezier edge geometry.

subgraph `module`

Subgraph namespace.

Subgraph objects are opaque zero-copy values. They can be passed to `graph.nodes` and `graph.edges`.

graph

- build()
- builder()
- cycles()
- dot()
- edge()
- edges()
- finish()
- forests()
- info()
- join()
- node()
- nodes()
- parse()

build

Build one graph object from node and edge arrays.

This is convenience sugar over `builder()`, `node()`, `edge()`, and `finish()`.

```
#let g = graph.build(
  name: "demo",
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.info(g).name
```

demo

Parameters

```
build(
  name: none string,
  statements: dictionary,
  edge-statements: dictionary,
  node-statements: dictionary,
  nodes: array,
  edges: array
) -> bytes
```

name `none` or `string`

Graph name.

Default: `none`

statements `dictionary`

Graph-level DOT statements.

Default: `(:)`

edge-statements `dictionary`

Default DOT statements for edges. String values may use `{name}` placeholders that expand from each edge's merged statement dictionary.

Default: `(:)`

node-statements `dictionary`

Default DOT statements for nodes.

Default: `(:)`

nodes `array`

Node specifications. Each node may define `name`, `index`, and `statements`. This matches the per-node `statements` parameter accepted by `node()`.

Default: `()`

edges `array`

Edge specifications. Each edge may define `source`, `sink`, `orientation`, `flow`, `id`, and `statements`. The `source` and `sink` fields use the same endpoint dictionaries accepted by `edge()`.

Default: `()`

builder

Create a builder object.

`statements`, `node-statements`, and `edge-statements` set graph-level DOT attributes that are carried into the finished graph. String values may use `{name}` placeholders; edge defaults expand after per-edge statements are merged, so an `edge-statements` value can refer to a `label` supplied by `edge()`.

```
#let b = graph.builder(
  name: "demo",
  statements: (full_num: "x + y"),
  edge-statements: (eval_label: "(text(fill:
rgb(\"#{color}\"))[{label}])"),
)
#let (node: a, builder: b) = graph.node(b, name:
"a")
#a
```

```
0
```

Parameters

```
builder(
  name,
  statements,
  node-statements,
  edge-statements
) -> bytes
```

cycles

Return subgraph objects for the graph's cycle basis.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.cycles(g).len()
```

```
0
```

Parameters

```
cycles(graph) -> array
```

dot

Serialize a graph object to DOT.

```
#let g = graph.build(
  name: "demo",
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.dot(g).contains("digraph demo")
```

```
true
```

Parameters

```
dot(graph) -> string
```

edge

Add a paired or external edge to a builder object.

Set `source` to `none` or `sink` to `none` for an external edge. Endpoint dictionaries accept `node`, `statement`, `id`, `port_label`, `compass`, and `in_subgraph`.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
```

```
"a")
#let (node: c, builder: b) = graph.node(b, name:
"c")
#let b = graph.edge(
  b,
  source: (node: a, compass: "e"),
  sink: (node: c, compass: "w"),
  statements: (label: "a-c"),
)
#graph.edges(graph.finish(b)).len()
```

```
1
```

Parameters

```
edge(
  builder,
  source,
  sink,
  orientation,
  flow,
  id,
  statements
) -> bytes
```

edges

Return edge records, optionally filtered by an subgraph object.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"),
sink: (node: 1)),),
)
#let east = subgraph.compass(g, "e")
#graph.edges(g, subgraph: east).len()
```

```
1
```

Parameters

```
edges(
  graph,
  subgraph
) -> array
```

finish

Finish a builder object into a graph object.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
"a")
#let g = graph.finish(b)
#graph.info(g).name
```

```
demo
```

Parameters

```
finish(builder) -> bytes
```

forests

Return subgraph objects for the graph's spanning forests.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.forests(g).len()
```

```
1
```

Parameters

```
forests(graph) -> array
```

info

Return graph metadata.

The result has `name`, `global_statements`, `edge_statements`, and `node_statements`.

```
#let g = graph.build(name: "demo", nodes:
((name: "a"),))
#graph.info(g).name
```

```
demo
```

Parameters

```
info(graph) -> dictionary
```

join

Join two graphs by matching dangling half-edge data on `key`.

Supported key values are `"statement"`, `"port_label"`, `"compass"`, and `"id"`.

```
#let left = graph.build(
  nodes: ((name: "a"),),
  edges: ((source: none, sink: (node: 0,
statement: "j")),),
)
#let right = graph.build(
  nodes: ((name: "b"),),
  edges: ((source: (node: 0, statement: "j"),
sink: none),),
)
#graph.edges(graph.join(left, right, key:
"statement")).len()
```

```
1
```

Parameters

```
join(
  left,
  right,
  key
) -> bytes
```

node

Add a node to a builder object.

Returns a dictionary with fields `node` and `builder`, so callers can write `#let (node: a, builder: b) = graph.node(b, name: "a")`. The returned builder is the value to pass to the next `node()`, `edge()`, or `finish()` call.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
"a")
#let (node: c, builder: b) = graph.node(b, name:
"c")
#repr((a, c))
```

```
(0, 1)
```

Parameters

```
node(
  builder,
  name,
  index,
  statements
) -> dictionary
```

nodes

Return node records, optionally filtered by an subgraph object.

```
#let g = graph.build(nodes: ((name: "a"), (name:
"b")))
#graph.nodes(g).map(node => node.name).join(",
")
```

```
a, b
```

Parameters

```
nodes(
  graph,
  subgraph
) -> array
```

parse

Parse one or more DOT digraphs into graph objects.

```
#let graphs = graph.parse("digraph first { a ->
b }")
#graphs.len()
```

```
1
```

Parameters

```
parse(input) -> array
```

curve

- cetz-args()
- cetz-bezier()
- cetz-edge-halves()
- cetz-pattern()
- coil()
- edge-halves()
- hobby-through()
- outset-point()
- pattern-cubic()
- pattern-segment()
- split-cubic()

- split-through()
- trim-cubic()
- trim-segment()
- wave()
- zigzag()

cetz-args

Convert a cubic segment returned by `split-cubic()` or `split-through()` into positional arguments accepted by `cetz.draw.bezier`.

Parameters

```
cetz-args(
  segment,
  unit
) -> array
```

cetz-bezier

Draw one cubic segment through CeTZ.

Parameters

```
cetz-bezier(
  segment,
  unit,
  ..style
) -> content
```

cetz-edge-halves

Draw the two halves of a laid-out graph edge through CeTZ.

`source-style` applies from the source node to the edge layout point, and `sink-style` applies from the edge layout point to the sink node.

Parameters

```
cetz-edge-halves(
  edge,
  nodes,
  unit,
  omega,
  source-outset,
  sink-outset,
  accuracy,
  source-style,
  sink-style
) -> content
```

cetz-pattern

Draw a sampled pattern returned by `pattern-cubic()` or `pattern-segment()`.

Parameters

```
cetz-pattern(
  path,
  unit,
  ..style
) -> content
```

coil

A smooth coil path pattern.

Parameters

```
coil(
  samples-per-period,
  longitudinal-scale
) -> dictionary
```

edge-halves

Split a laid-out graph edge into source and sink half-edge cubic segments.

The returned dictionary has `source`, `sink`, and `curve`. The split point is the edge layout point, so the two half-edges join smoothly there.

Parameters

```
edge-halves(
  edge,
  nodes,
  omega,
  source-outset,
  sink-outset,
  accuracy
) -> dictionary
```

hobby-through

Build an open Hobby curve through `start`, `through`, and `end`.

Returns a dictionary with two cubic Bezier `segments`, split at `through`.

Parameters

```
hobby-through(
  start,
  through,
  end,
  omega
) -> dictionary
```

outset-point

Move `from` toward `toward` by `distance`, capped before `toward`.

Parameters

```
outset-point(
  from,
  toward,
  distance
) -> dictionary
```

pattern-cubic

Generate a sampled 1D pattern along a cubic Bezier path.

`pattern` may be `wave()`, `zigzag()`, `coil()`, a compatible string name, or a point pattern: `(kind: "points", interpolation: "linear" or "smooth", points: ((at: 0, x: 0, y: 0), ...))`. The returned dictionary contains `points`, smooth `curves` for smooth patterns, and straight `segments` in graph coordinates.

Parameters

```
pattern-cubic(
  start,
  end,
  ctrl-a,
  ctrl-b,
  pattern,
  amplitude,
  wavelength,
  phase,
  samples-per-period,
  coil-longitudinal-scale,
  anchor-start,
  anchor-end,
  accuracy
) -> dictionary
```

pattern-segment

Generate a sampled 1D pattern along a cubic segment returned by this module.

Parameters

```
pattern-segment(
  segment,
  pattern,
  amplitude,
  wavelength,
  phase,
  samples-per-period,
  coil-longitudinal-scale,
  anchor-start,
  anchor-end,
  accuracy
) -> dictionary
```

split-cubic

Split a cubic Bezier curve into two cubic Bezier segments.

Returns a dictionary with `curve` and `segments`. Each segment has `start`, `ctrl_a`, `ctrl_b`, and `end` points.

Parameters

```
split-cubic(
  start,
  end,
  ctrl-a,
  ctrl-b,
  t
) -> dictionary
```

split-through

Build a quadratic Bezier through `start`, `through`, and `end`, promote it to a cubic, then split it at `t`.

This is useful for graph edges whose layout point should lie on the smooth curve between source and sink.

Parameters

```
split-through(
  start,
  through,
  end,
  t
) -> dictionary
```

trim-cubic

Trim a cubic Bezier by curve distance from either end.

Returns the remaining cubic segment after splitting by arc length.

Parameters

```
trim-cubic(
  start,
  end,
  ctrl-a,
  ctrl-b,
  start-outset,
  end-outset,
  accuracy
) -> dictionary
```

trim-segment

Trim a cubic segment returned by `split-cubic()`, `split-through()`, or `hobby-through()` by curve distance from either end.

Parameters

```
trim-segment(
  segment,
  start-outset,
  end-outset,
  accuracy
) -> dictionary
```

wave

A smooth sinusoidal path pattern.

Parameters

`wave(samples-per-period) -> dictionary`

zigzag

A straight-segment triangular path pattern.

Parameters

`zigzag() -> dictionary`

draw

- draw()

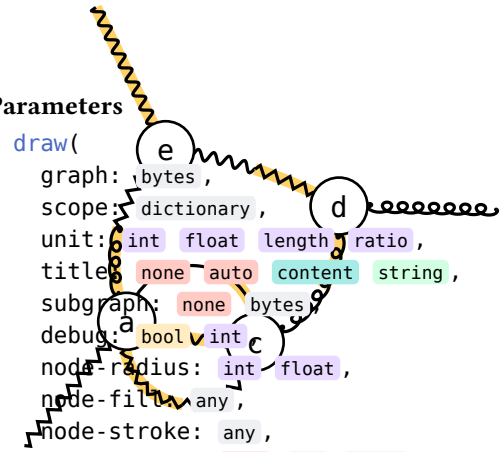
draw

Draw a laid-out graph object with CeTZ.

The graph must already have positions, so call `layout` first. Node and edge DOT `eval_*` statements are evaluated in `scope` and forwarded to CeTZ.

```
#let b = graph.builder(
  name: "demo",
  edge-statements: (
    eval_source: "(stroke: black + 1.2pt,
pattern: source-patterns.at(eid), pattern-
amplitude: 0.18, pattern-wavelength: 0.55,
pattern-coil-longitudinal-scale: 1.6)",
    eval_sink: "(stroke: black + 1.2pt, pattern:
sink-patterns.at(eid), pattern-amplitude: 0.18,
pattern-wavelength: 0.55, pattern-coil-
longitudinal-scale: 1.6)",
  ),
)
#let source-patterns = (
  none,
  "wave",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
)
#let sink-patterns = (
  none,
  "wave",
  "zigzag",
  "coil",
  "zigzag",
  "coil",
  "wave",
  "coil",
  "wave",
)
#let (node: a, builder: b) = graph.node(b, name:
"a")
#let (node: c, builder: b) = graph.node(b, name:
"c")
#let (node: d, builder: b) = graph.node(b, name:
"d")
```

```
#let (node: e, builder: b) = graph.node(b, name:
"e")
#let b = graph.edge(b, source: (node: a), sink:
(node: c, compass: "e"))
#let b = graph.edge(b, source: (node: a,
compass: "e"), sink: (node: c,  compass: "e"))
#let b = graph.edge(b, source: (node: c), sink:
(node: a, compass: "e"))
#let b = graph.edge(b, source: (node: c), sink:
(node: d, compass: "e"))
#let b = graph.edge(b, source: (node: e), sink:
(node: d, compass: "e"))
#let b = graph.edge(b, source: (node: e), sink:
(node: a, compass: "e"))
#let b = graph.edge(b, source: (node: d), sink:
none)
#let b = graph.edge(b, source: (node: e,
compass: "e"), sink: none)
#let b = graph.edge(b, source: (node: a), sink:
none)
#let layed-out = layout(graph.finish(b), beta:
50,gamma-ee:0.1,gamma-dangling:5,gamma-
ev:0.01,seed: 2,epochs:30, steps: 30, g-center:
0.005, length-scale: 0.5)
#let east = subgraph.compass(layed-out, "e")
#draw(layed-out,node-radius:
1,subgraph:east,scope: (source-patterns: source-
patterns, sink-patterns: sink-patterns))
```



Parameters

```
draw(
  graph: bytes,
  scope: dictionary,
  unit: int | float | length | ratio,
  title: none | auto | content | string,
  subgraph: none | bytes,
  debug: bool | int,
  node-radius: int | float,
  node-fill: any,
  node-stroke: any,
  node-outset: auto | int | float,
  node-label-style: dictionary,
  edge-stroke: any,
  edge-omega: float,
  edge-trim-accuracy: float,
  padding: none | int | float | array | dictionary,
  debug-edge-radius: int | float,
  debug-edge-fill: any,
  debug-edge-stroke: any,
  debug-edge-label-fill: any,
  subgraph-edge-style: dictionary,
  subgraph-edge-underlay: bool
) -> content
```

graph `bytes`

Laid-out graph object returned by `layout`.

scope `dictionary`

Additional symbols available while evaluating DOT `eval_*` statements.

Default: `(:)`

unit `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: 1

title `none` or `auto` or `content` or `string`

Optional title displayed above the diagram. Use `auto` for the graph name.

Default: none

subgraph `none` or `bytes`

Optional subgraph whose half-edges are shaded.

Default: none

debug `bool` or `int`

Debug level. `1` enables CeTZ canvas debug; `2` also marks edge positions.

Default: `false`

node-radius `int` or `float`

Default CeTZ node radius.

Default: `0.16`

node-fill `any`

Default CeTZ node fill.

Default: `white`

node-stroke `any`

Default CeTZ node stroke.

Default: `black`

node-outset `auto` or `int` or `float`

Edge clearance from node centers. `auto` uses each node circle radius. Increase this when labels extend beyond the circle.

Default: `auto`

node-label-style `dictionary`

Default node-label style forwarded to `cetz.draw.content`.

Default: `(:)`

edge-stroke `any`

Default CeTZ edge stroke.

Default: `0.1em`

edge-omega `float`

Hobby curl used at the endpoints of paired edge curves.

Default: `1.0`

edge-trim-accuracy `float`

Arc-length accuracy for trimming edge curves at node outsets.

Default: `0.001`

padding `none` or `int` or `float` or `array` or `dictionary`

CeTZ canvas padding.

Default: `0.4`

debug-edge-radius `int` or `float`

Radius for edge-position markers shown at `debug >= 2`.

Default: `0.08`

debug-edge-fill `any`

Fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#ff9f1c")`

debug-edge-stroke `any`

Stroke for edge-position markers shown at `debug >= 2`.

Default: `rgb("#d72638") + 0.35pt`

debug-edge-label-fill `any`

Label fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#7a1020")`

subgraph-edge-style `dictionary`

CeTZ style used to shade half-edges included in `subgraph`.

Default: `(stroke: rgb("#ffd166") + 4.5pt)`

subgraph-edge-underlay `bool`

Draw subgraph shading below the normal half-edge style.

Default: `true`

subgraph

- bits()
- compass()
- contains()
- hedges()
- label()
- to-label()

bits

Construct an subgraph object from a boolean hedge array.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
```

```
)
#let first = subgraph.bits(g, (true, false))
#subgraph.contains(first, 0)
```

```
true
```

Parameters

```
bits(
  graph,
  bits
) -> bytes
```

compass

Construct an subgraph object from a DOT compass point such as `"n"` or `"s"`.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"),
sink: (node: 1)),),
)
#subgraph.hedges(subgraph.compass(g, "e")).len()
```

```
1
```

Parameters

```
compass(
  graph,
  compass
) -> bytes
```

contains

Test whether an subgraph object includes a hedge.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#subgraph.contains(subgraph.bits(g, (true,
false)), 0)
```

```
true
```

Parameters

```
contains(
  subgraph,
  hedge
) -> bool
```

hedges

Return the hedge indices included in an subgraph object.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
```

```
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#subgraph.hedges(subgraph.bits(g, (true,
false)))
```

```
(0,)
```

Parameters

`hedges(subgraph) -> array`

label

Construct an subgraph object from a base62 label.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"),
sink: (node: 1)),),
)
#let east = subgraph.compass(g, "e")
#let same = subgraph.label(g, subgraph.to-
label(east))
#subgraph.hedges(same).len()
```

```
1
```

Parameters

```
label(
  graph,
  label
) -> bytes
```

to-label

Convert an subgraph object to its base62 label.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#subgraph.to-label(subgraph.bits(g, (true,
false)))
```

```
1
```

Parameters

`to-label(subgraph) -> string`